



SE423

SOFTWARE CONSTRUCTION AND DEVELOPMENT

MicroLend

Community Microfinance Loan Servicing and Delinquency Engine

MILESTONE 1 · WEEK 6

Proposal, Object-Oriented Design & Pattern Plan

TEAM FABLE

Muhammad Ibrahim

TEAM LEAD

2023446 · origination · delinquency

Hassan Khalid

2023242 · products · reporting

Tughral Hussain

2023532 · ledger · web layer

INSTRUCTOR

Dr. Mian Muaz Razaq

REPOSITORY

github.com/tughral1/microlend

Contents

1	Problem Definition and Requirement-to-Minimum Mapping	3
2	Module Breakdown and Ownership	5
3	Object-Oriented Design	6
4	SOLID Application Plan	15
5	Design Pattern Selection and Justification	17
6	Repository and Documentation Setup	19
Annex A — AI Usage Log		22
	Declaration for Milestone 1	23
	Team rules for AI use	23

1 Problem Definition and Requirement-to-Minimum Mapping

1.1 The system

A community microfinance institution lends small amounts — typically Rs. 25,000 to Rs. 200,000 — to shopkeepers, tradespeople and smallholders who cannot access bank credit. The loans are short, repayment is frequent, and the institution's survival depends on knowing, every single day, exactly how much is owed, how much is overdue, and how much of the portfolio is at risk of never being recovered.

MicroLend is the back-office engine that answers those questions. It assesses loan applications against eligibility rules, generates repayment schedules, applies incoming repayments to the right obligations in the right order, maintains an audit-grade ledger, and runs a nightly process that ages overdue accounts, accrues penalties and classifies the portfolio by risk.

The system is a **back-office engine with a small web interface that runs locally**. It is never deployed: there is no hosting, no external service and no paid resource of any kind. The application serves its pages from a local process against a file-based database, so every demonstration runs entirely on a team member's own laptop.

1.2 Its users

User	What they do with MicroLend
Loan officer	Enters an application; the system approves or rejects it with a stated reason, and produces the repayment schedule the customer signs
Teller / cashier	Posts repayments as customers pay; the system decides how each payment is applied and records it
Branch manager	Reads portfolio reports — how much is overdue, how much must be provisioned, how effective collection has been
System (scheduled)	Runs the end-of-day process: accrues penalties, ages accounts, re-classifies risk, writes off unrecoverable loans

1.3 Why this problem is worth building software for

Three properties make this domain demanding in a way a CRUD application is not:

- 1. Money must be exactly right.** A repayment schedule whose instalments do not sum to principal plus interest is not "approximately correct"; it is wrong, and the error compounds across a portfolio.
- 2. Order of application matters.** The same Rs. 4,000 applied penalty-first versus principal-first produces different balances, different interest, and a different delinquency classification months later.
- 3. The record must be trustworthy.** A ledger that can be edited is not a ledger. Corrections must be visible as corrections.

Those three properties are what give the system its invariants, and the invariants are what the design in Section 3 protects.

1.4 Scope boundary

In scope for this project: individual loans, three product types, fixed-rate lending in a single currency, repayment by cash at the branch or by bank statement import, penalty accrual, delinquency classification, provisioning, write-off, statements and portfolio reporting.

Explicitly deferred (recorded here so that scope cannot creep silently): group/joint-liability lending, guarantors and collateral registers, loan restructuring and rescheduling, multi-currency, savings products, interest on savings, mobile money integration, and any deployment of the application beyond running it locally.

1.5 Mapping of each Section 3.1 minimum requirement to how it will be met

#	Minimum requirement (guidelines §3.1)	How MicroLend meets it	Evidence at
1	≥ 4 distinct functional modules with real business logic, not a single-file CRUD demo	Five modules: <code>origination</code> (eligibility decisions), <code>products</code> (schedule generation), <code>ledger</code> (allocation waterfall and posting), <code>delinquency</code> (penalty accrual, ageing, provisioning), <code>reporting</code> (statements and portfolio metrics). Each owns a distinct decision , not a screen — see §2	M2, M3
2	OO design demonstrating SOLID , with written justification of where and why each principle was applied	§3 gives the class design with cohesion and coupling reasoning; §4 analyses all five principles, identifying where each is relevant, not relevant, or not yet observable	M1 (§3, §4)
3	≥ 3 correctly implemented, justified patterns , at least one creational plus one structural or behavioural	Factory Method (creational) in <code>products</code> , Adapter (structural) at the bank-statement import boundary, Decorator (structural) on the repayment-posting pipeline. Each with the alternative rejected — §5	M1 plan, M2 implementation
4	≥ 1 genuine concurrent or asynchronous operation with a documented thread-safety strategy	The end-of-day engine processes the active portfolio across a fixed thread pool while tellers post repayments to the same accounts. Strategy in §3.6	M2
5	Defensive programming and structured exception handling across module boundaries	Validation in constructors and command factories so invalid objects cannot exist; a typed exception hierarchy rooted at <code>LoanDomainException</code> ; <code>Money</code> rejects invalid values at construction — §3.5	M2
6	Persistent data layer with a defined schema	SQLite, seven tables with foreign keys and <code>CHECK</code> constraints that re-state the code's invariants at the storage layer — §3.7	M2
7	TDD suite ≥ 60% of core business logic with visible red-green-refactor history	Eligibility, schedule generation, allocation and classification are pure functions over <code>Money</code> and dates — ideal test-first targets. Commit convention <code>test(red):</code> → <code>feat(green):</code> → <code>refactor:</code> ; squash-merge disabled in repository settings so the trail survives — §6.3	Repository history, M2, M3
8	Hosted Git repository with individually attributable contribution history	GitHub, public. Module ownership maps one-to-one onto directories, so <code>git log</code> per path is meaningful. <code>CODEOWNERS</code> requests review from the module owner; no author merges their own work into <code>main</code> — §6	Repository, all milestones

- | | | | |
|---|--|---|---|
| 9 | CI pipeline that builds and runs the test suite on push | GitHub Actions: compile, JUnit, JaCoCo coverage report, Checkstyle gate. No secrets, no network dependency — §6.4 | M1 (pipeline already green on <code>develop</code>), M3 (multiple genuine pipeline runs) |
|---|--|---|---|

2 Module Breakdown and Ownership

Five modules, one more than the minimum of four.

#	Module	Scope in one line	Primary owner
1	<code>origination</code>	Decides whether an application is approved, on what terms, and with what stated reason if rejected	Muhammad Ibrahim
2	<code>products</code>	Holds the loan product catalogue and generates the repayment schedule for an approved loan	Hassan Khalid
3	<code>ledger</code>	Applies incoming repayments through the allocation waterfall and records every movement in an append-only ledger	Tughral Hussain
4	<code>delinquency</code>	Runs the end-of-day process: accrues penalties, computes days past due, classifies and provisions accounts, writes off	Muhammad Ibrahim
5	<code>reporting</code>	Produces customer statements, portfolio risk reports and CSV exports	Hassan Khalid

Two supporting packages carry no business rules of their own but still have a named owner: the thin `web` layer and the composition root that wires implementations together (**Tughral Hussain**), and the bank-statement import adapters in `ledger.importing` (**Muhammad Ibrahim**, who owns the Adapter pattern — §5.2).

2.1 The business rules each module owns

These are frozen at M1 in the **Business Rules Register** (`docs/business-rules.md`) so that the requirements the SQA side traces against do not move underneath it.

origination — An applicant must be 18 or older at disbursement and no older than 65 at maturity; may hold at most 2 active loans, counting the one applied for; total monthly instalment burden must not exceed 40% of declared monthly income; must have no prior write-off; the requested amount and tenure must lie within the product's bands. A failing application yields a **typed rejection reason**, not a boolean. Risk grade A/B/C — assigned from the burden ratio and the applicant's past repayment record — adjusts the rate by a fixed spread and caps the approved amount.

products — Three products: **flat-rate** (interest on the original principal throughout), **reducing-balance** (interest on the outstanding balance, equal instalments), and **bullet** (interest monthly, principal at maturity). Instalment dates follow the same day-of-month as disbursement, clamped for short months. An optional grace period defers the first instalment but still accrues interest. All rounding residue is absorbed into the final instalment, so the instalments sum **exactly** to principal plus total interest.

ledger — Payments are applied in strict order: **penalty** → **fees** → **interest** → **principal**, oldest unpaid instalment first (no fee type is defined at M1, so the fees tier is always zero until one is introduced by a recorded deviation). Partial payments are allowed and applied as far as they reach. An overpayment becomes an advance credit, unless it clears the loan, in which case early settlement applies: unearned interest is rebated for reducing-balance loans, unaccrued monthly interest for bullet loans, and no rebate for flat-rate loans. **A posted ledger entry is immutable**; a mistake is corrected only by a compensating reversal entry.

delinquency — For a given business date: accrue a penalty of 0.05% per day on overdue principal, capped at 10% of the overdue amount; compute days past due; place the account in a bucket (0 / 1–30 / 31–60 / 61–90 / 91+ days) mapping to a classification (Standard / Watch / Substandard / Doubtful / Loss); apply the matching provision percentage; write the loan off automatically past 180 days overdue.

reporting — Customer statement for a date range; portfolio reports (portfolio at risk over 30 days, provision coverage, collection efficiency); CSV export; an alert event raised whenever an account's classification changes.

2.2 Why ownership is split this way

Each owner holds a module whose **invariant fits in one sentence**:

- **Muhammad Ibrahim** — *"No loan may exist that the eligibility rules would not have approved."*
- **Hassan Khalid** — *"The instalments of a schedule sum exactly to principal plus total interest."*
- **Tughral Hussain** — *"The ledger always balances, and nothing in it is ever altered."*

Ibrahim additionally owns the concurrent end-of-day engine, which depends on Tughral's ledger; that dependency is deliberate, because it forces the two of them to agree an interface rather than reach into each other's internals.

SE431 deliverable ownership rotates across the three milestones, so every member leads each kind of quality artifact once and reviews the others each time:

Milestone	SQAP, V&V and risk register	Requirements, RTM and test design	Inspection, analysis and measurement
M1 (Week 6)	Muhammad Ibrahim — SQAP scope and outline, initial risk register	Hassan Khalid — quality requirements, RTM v1	Tughral Hussain — standards awareness, Cost of Quality
M2 (Week 10)	Hassan Khalid — V&V plan (IEEE 1012), risk register update	Tughral Hussain — black-box and white-box test design, RTM update	Muhammad Ibrahim — Fagan inspection and CAPA, static analysis, SCM quality-gate evidence
M3 (Week 13)	Tughral Hussain — final IEEE 730 SQAP, final risk register	Muhammad Ibrahim — final test execution and coverage, final RTM	Hassan Khalid — GQM programme, final audit and CAPA closure

3 Object-Oriented Design

3.1 Design intent

The design is organised around one idea: **the objects that hold money-related state are responsible for defending their own correctness**. Validation is not performed by services on passive data structures; it happens inside the objects, at construction and at every state transition, so that an invalid object cannot exist for other code to defend against.

Everything else follows from that. Services are thin and stateless. Data crosses module boundaries as immutable value objects. Persistence is reached only through interfaces the domain itself declares.

3.2 Class diagram

The class model is shown in four views, one per area of the system, and every design element named in the RTM appears in one of them. Shared kernel types (`Money` , `LoanId` , the exception hierarchy) appear where they are used.

DIAGRAM

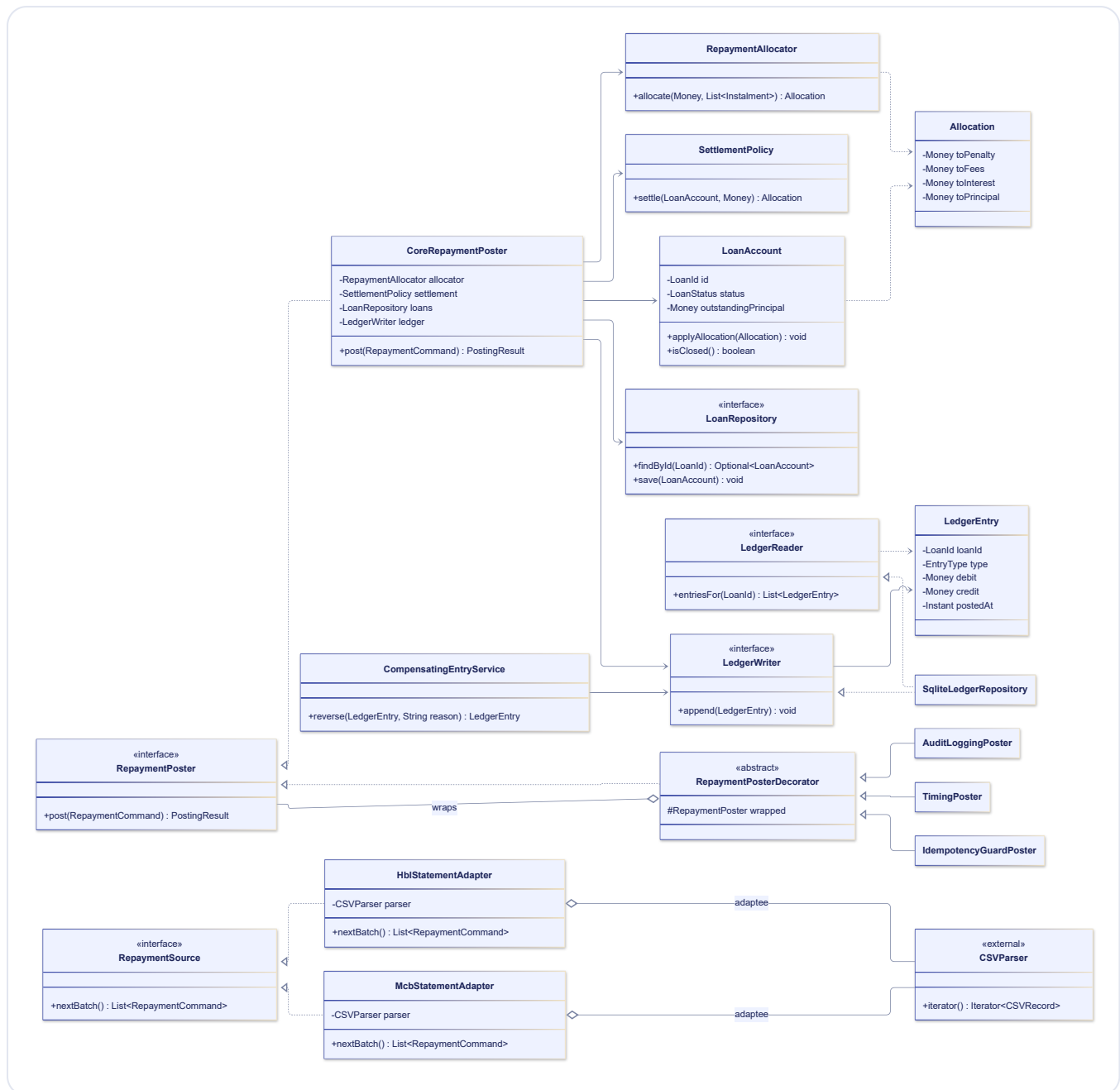
View 1 — Origination and Products: deciding whether to lend, and on what schedule



Note the two extension points: a new eligibility rule is a new `EligibilityRule`; a new loan product is a new `LoanProduct` subclass with its calculator. Neither requires editing a tested class. `DueDateCalculator`, `GracePeriodPolicy` and `ScheduleAssembler` are the fixed steps of the `generateSchedule()` workflow; only the interest calculation varies by product (§5.1).

DIAGRAM

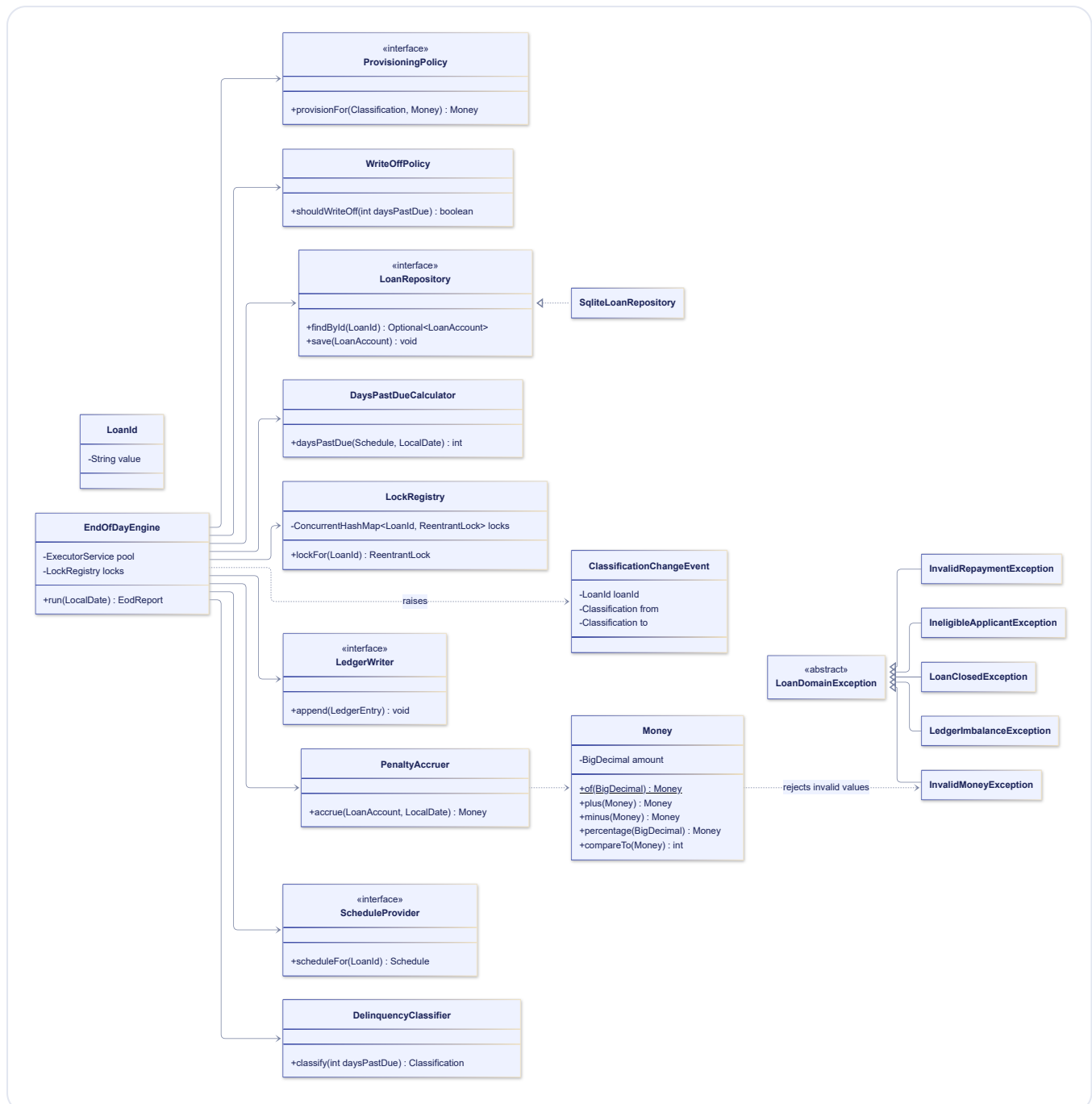
View 2 — Ledger: applying money, with the posting pipeline and the import boundary



The Decorator chain and the Adapter boundary are both visible here: decorators are each IS-A and HAS-A `RepaymentPoster`, and each bank adapter holds the third-party `CSVParser` (Apache Commons CSV) by composition rather than inheriting it. The ledger's persistence boundary is two role interfaces, `LedgerWriter` and `LedgerReader`, implemented by one SQLite class (§4.4).

DIAGRAM

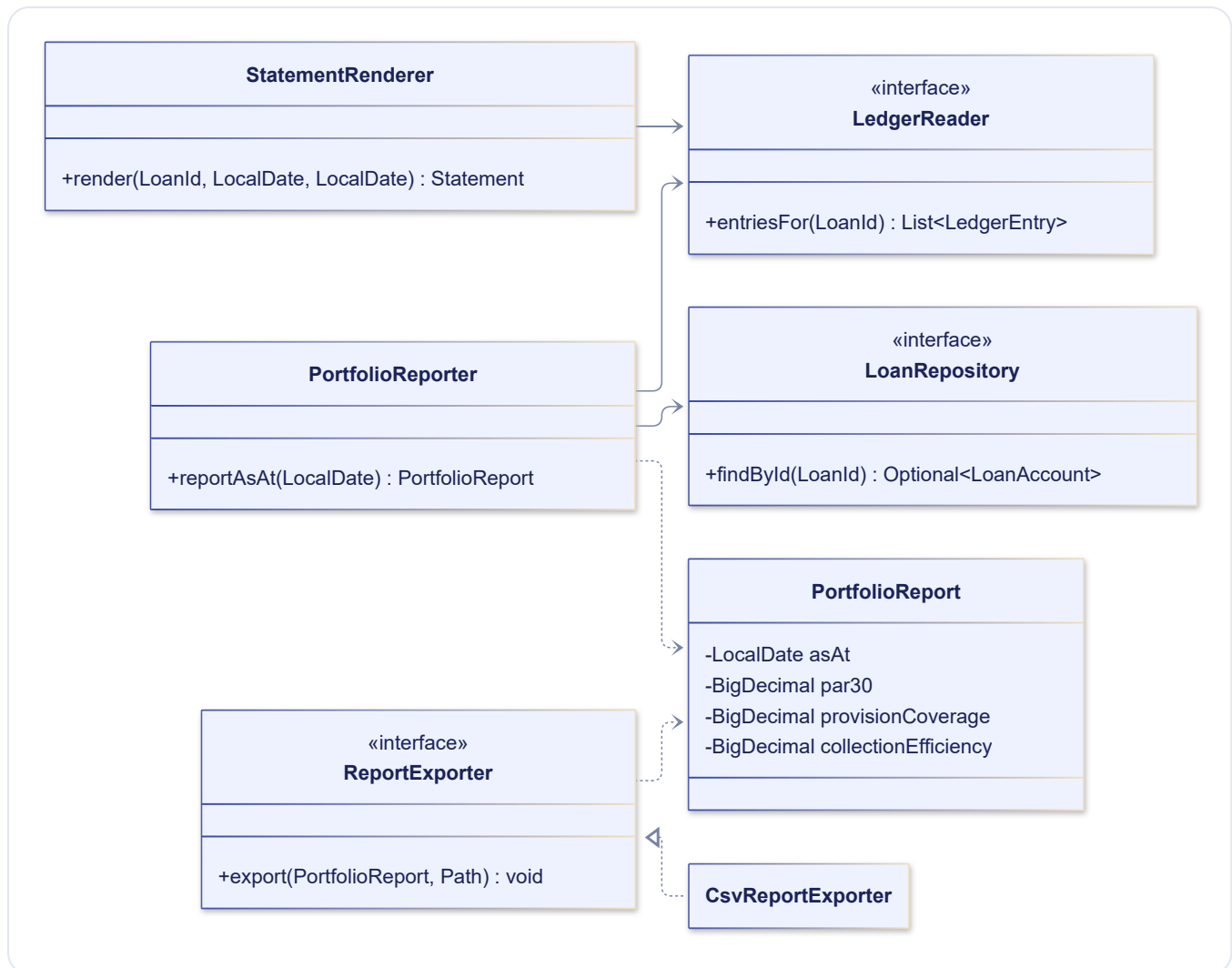
View 3 — Delinquency, the shared kernel, and the persistence boundary



`EndOfDayEngine` holds only the interfaces it needs — `LoanRepository`, `ScheduleProvider` and `LedgerWriter` — so it cannot export a report or read the full ledger, because those methods are not on the types it is given. The exception hierarchy is rooted at `LoanDomainException`, so nothing outside a module ever catches a storage-layer exception.

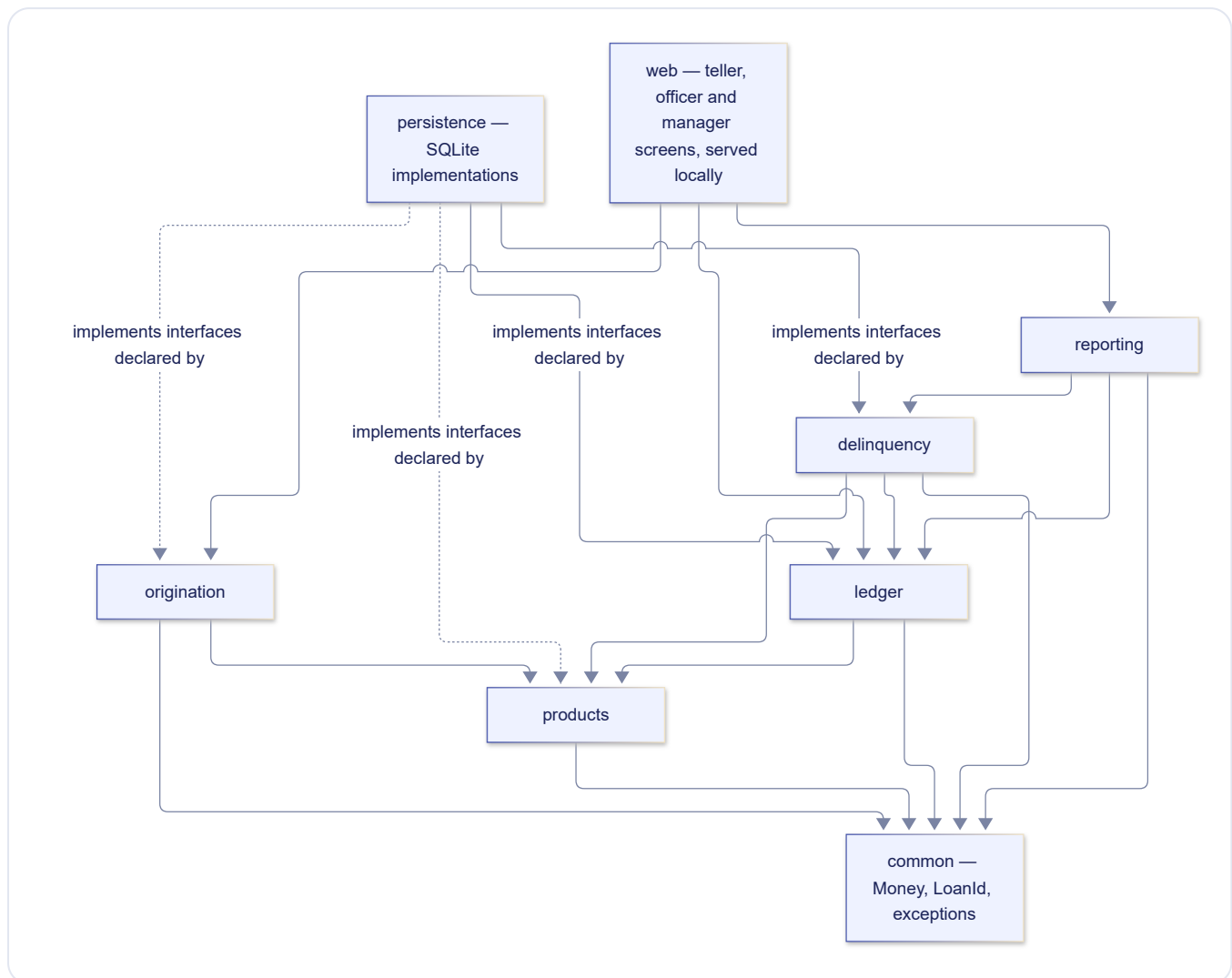
DIAGRAM

View 4 — Reporting: presenting what has already happened



`reporting` only reads: it holds `LedgerReader` and `LoanRepository`, never `LedgerWriter`, so a report cannot change state. Every figure is computed as at a stated business date (BR-39), which is what makes a report reproducible for any past date.

3.3 Component view and allowed dependencies



The dependency rule: arrows point inward, toward `common`, and never outward from a domain module to persistence or the web layer. `persistence` depends on the domain, not the reverse — the dashed arrows are implementations of interfaces the domain declares. There are **no cycles** between modules (constraint C-03). From M2 this is enforced by an ArchUnit test that runs in CI with the rest of the suite, rather than left as an intention.

3.4 Cohesion and coupling reasoning

Cohesion — each module is functionally cohesive, not grouped by convenience.

The test applied to every module was: *do these operations exist to serve one job, or do they merely share a noun?* A class holding `assessEligibility()`, `saveToDatabase()`, `renderStatementPdf()` and `emailCustomer()` would be only **logically cohesive** — grouped because each mentions a loan — and would answer to four different stakeholders. That class does not exist here. Instead:

Module	Its single job	What was deliberately kept out
<code>origination</code>	Decide approval and terms	Schedule arithmetic (that is <code>products</code>), persistence (that is an injected repository)
<code>products</code>	Turn an approved amount and tenure into a schedule	Deciding <i>whether</i> to lend, and anything about repayment

<code>ledger</code>	Apply money to obligations and record the movement	Deciding penalties or classification (that is <code>delinquency</code>), formatting output
<code>delinquency</code>	Age, penalise, classify and provision	Writing ledger lines itself — it asks <code>ledger</code> to do that
<code>reporting</code>	Present what has already happened	Any calculation that changes state

Within modules the same test is applied to classes. `EligibilityEvaluator` holds no rules of its own; each rule is a separate `EligibilityRule` implementation, so adding “must not be a staff member’s relative” adds a class rather than an `if`. `RepaymentAllocator` computes an `Allocation` and does not write it; `CoreRepaymentPoster` writes it and does not compute it.

Coupling — loose, and in one direction.

- 1. No module constructs another module’s dependencies.** Business classes never execute `new SQLiteLoanRepository(...)`. Repositories arrive through constructors, so `ledger` depends on the `LedgerWriter` and `LedgerReader` interfaces it declares itself, and the SQLite implementation depends inward on those interfaces. The effect is concrete: the posting logic can be tested with an in-memory fake, with no database on the machine, and swapping SQLite for PostgreSQL changes one class in `persistence` and nothing in any domain module.
- 2. Data crosses boundaries as immutable value objects.** `Money`, `Instalment`, `LedgerEntry` and `Allocation` have no setters. A caller cannot mutate another module’s state by holding a reference to something it was given, and no module needs defensive copying on receipt.
- 3. No internal collection is ever returned.** `Schedule.instalments()` returns an unmodifiable view. Returning the live list would let a caller bypass every rule in `Schedule`, and would freeze the choice of `List` as a public API that could never be changed to a `Map` for lookup by due date.
- 4. Content coupling is impossible by construction.** No module exposes mutable fields; all state is private and changed only through methods that enforce the invariant.
- 5. The one accepted dependency with weight** is `delinquency → ledger`, because penalty accrual must write ledger entries. It is deliberately **interface-only**: the end-of-day engine holds a `LedgerWriter`, not the posting implementation, and cannot read statements or export CSV through it.

Where coupling is highest, and why that is acceptable: `products` and `ledger` are the most depended-upon domain modules — `origination`, `ledger` and `delinquency` need the schedule from `products`, and `delinquency` and `reporting` need the ledger. This is *afferent* coupling — many things depend on them — which suits stable modules whose contracts (a schedule; an append-only list of entries) are small and unlikely to change. The risk it creates is that a defect in either propagates to every downstream figure. That risk is recorded as **R1** and mitigated by the golden-schedule tests and the basis-path tests on the allocation waterfall, both written before the implementation.

3.5 Defensive programming and exception design

Invalid objects cannot be constructed.

```

public final class Money implements Comparable<Money> {
    public static final int SCALE = 2;
    private final BigDecimal amount;

    private Money(BigDecimal normalisedAmount) { this.amount = normalisedAmount; }

    public static Money of(BigDecimal value) {
        if (value == null)
            throw new InvalidMoneyException("amount is required");
        if (value.scale() > SCALE)
            throw new InvalidMoneyException("more than 2 decimal places: " + value);
        if (value.signum() < 0)
            throw new InvalidMoneyException("amount may not be negative: " + value);
        return new Money(value.setScale(SCALE, RoundingMode.HALF_EVEN));
    }
}

```

This is an abridged excerpt (messages shortened) of the `Money` class already committed and tested in the repository (`common/Money.java` , `MoneyTest`). The constructor is private, so `Money.of()` is the only way in, and every rejection is an `InvalidMoneyException` — a `LoanDomainException` subtype. A single currency (PKR, BR-41) is assumed, so `Money` carries no currency field. Because `Money` cannot hold a negative or over-scaled value, no downstream code tests for one. The same applies to `LoanApplication` (validated in its constructor) and `RepaymentCommand` (validated in its factory). This is the difference between defending against bad input at one boundary and scattering `if (amount < 0)` through five modules.

A typed exception hierarchy, carrying the reason.

```

LoanDomainException (abstract — the module boundary type)
├─ IneligibleApplicantException (carries the failing RuleOutcome)
├─ InvalidRepaymentException (carries the offending value and its source)
├─ LoanClosedException (carries loan id and closure date)
├─ LedgerImbalanceException (carries the computed debit and credit totals)
└─ InvalidMoneyException

```

Rules: no `catch (Exception e)` , and no empty catch blocks (unless the exception variable is named `expected` or `ignored` , which marks a deliberate test case) — both are rejected by the static-analysis configuration. Exceptions crossing a module boundary are always `LoanDomainException` subtypes, so a caller never catches a JDBC exception; the repository translates storage failures into domain terms. Each exception carries the data needed to explain the failure, because `throw new RuntimeException("invalid")` tells an operator nothing.

3.6 The concurrent operation and its thread-safety strategy

What runs concurrently. The end-of-day engine processes the active portfolio across `Executors.newFixedThreadPool(4)` , one task per loan account, **while teller threads are still posting repayments to the same accounts.** In a real branch the nightly batch and late counter transactions overlap, and the demo harness runs both at once.

The three races it must prevent:

- 1. Lost update.** A teller reads an outstanding balance of 10,000; the end-of-day task reads 10,000; both write back. One movement vanishes and the ledger stops balancing.
- 2. Double penalty accrual.** If the engine runs twice for the same business date — a retry after failure, or two pool threads taking the same account — penalty is charged twice, inventing money.
- 3. Torn read.** A portfolio report sums balances mid-update and reports a total that was never true at any instant.

The strategy:

Measure	What it prevents
Immutability at the core — <code>Money</code> , <code>LedgerEntry</code> , <code>Instalment</code> have no mutable state	Nothing shared can be changed under another thread
Per-account locks — <code>LockRegistry</code> holds a <code>ConcurrentHashMap<LoanId, ReentrantLock></code> . Each account's read → allocate → write sequence runs under that account's lock and inside one SQLite transaction (<code>BEGIN IMMEDIATE</code> , with a busy timeout and one connection per worker), committed before the lock is released	Lost updates, while unrelated accounts are computed in parallel
One lock at a time — no task ever holds two account locks; nothing in scope moves money between accounts	Deadlock, which needs at least two locks held at once to form a cycle
Unique (<code>loan_id</code> , <code>business_date</code>) in <code>delinquency_snapshot</code> , inserted in the same transaction as that day's penalty ledger entries	Double accrual — a second run for the same date fails on the key and its whole transaction, accrual included, rolls back
Append-only ledger , with an invariant test asserting $\Sigma \text{ debits} = \Sigma \text{ credits}$ after every run	Silent imbalance under any interleaving
Snapshot reads — a report runs inside one SQLite read transaction (WAL mode gives it a consistent snapshot) and computes as at a stated business date from the append-only ledger (BR-39)	Torn read — a report never sees half of a posting
Deterministic tests driven by <code>CountDownLatch</code> , never <code>Thread.sleep</code>	Flaky CI, which is what causes teams to start ignoring red builds

A stated limit. SQLite admits one writer at a time, so the four pool threads run the *calculation* (days past due, penalty, classification, provisioning) in parallel, while the commits themselves are short and serialised by the database. QR-01 in the SE431 document measures whether that is fast enough for 10,000 loans, rather than assuming it.

The same design gives the two injected failure scenarios planned for M3: a repository that throws mid-posting must leave the ledger balanced, and an end-of-day task that dies must not block the pool or leave a half-applied accrual.

3.7 Persistent data layer

SQLite, seven tables. The schema **re-states the code's invariants** so that a corrupt write fails at two independent levels (foreign keys are enforced with `PRAGMA foreign_keys = ON` on every connection, because SQLite leaves them off by default):

Table	Holds	Constraints that matter
<code>customer</code>	Applicant identity, income, write-off history	—
<code>loan_product</code>	Catalogue: code, rate, amount band, tenure band	<code>CHECK (min_amount ≤ max_amount)</code>
<code>loan</code>	Account state: principal, status, grade, dates	FK to customer and product; <code>CHECK (outstanding ≥ 0)</code>
<code>instalment</code>	Generated schedule lines	FK to loan; <code>UNIQUE (loan_id, sequence)</code>
<code>ledger_entry</code>	Every money movement	FK to loan; <code>CHECK (debit ≥ 0 AND credit ≥ 0)</code> ; no UPDATE or DELETE statement against this table exists in the codebase

<code>payment</code>	Received payments and their source	<code>UNIQUE (source_bank, bank_reference)</code> — the idempotency guard; cash payments store <code>NULL</code> , which SQLite treats as distinct
<code>delinquency_snapshot</code>	End-of-day result per loan per date	<code>UNIQUE (loan_id, business_date)</code> — prevents double accrual

Monetary columns are stored as `INTEGER` minor units (paisa) rather than `REAL`, for the same reason `double` is banned in the code.

4 SOLID Application Plan

All five principles are relevant to MicroLend; none is declared not applicable. Each is tied below to a concrete design decision. Four (SRP, OCP, ISP, DIP) are demonstrated by decisions already visible in the class views; LSP is designed in but **not yet observable** until its contract test runs against real calculators at M2, and §4.6 says what evidence will show it.

4.1 Single Responsibility Principle — relevant, demonstrated in the design

The test applied: who can demand a change to this class? If two different stakeholders can, it has two reasons to change.

The design decision this produced: an early sketch had one `LoanService` holding eligibility checks, schedule generation, repayment posting and statement rendering. Four actors could demand changes to it — the credit committee (eligibility), the finance department (rates and schedules), the branch operation (how payments are applied) and the marketing/communications side (statement layout). It was split into `EligibilityEvaluator`, `LoanProduct` (with `ScheduleAssembler`), `RepaymentAllocator` and `StatementRenderer`.

The concrete consequence: a change to the penalty rate touches `delinquency` alone; a change to statement layout touches `reporting` alone. Neither forces re-testing of the repayment waterfall. Had they stayed together, a statement-formatting change would require re-certifying money-handling code.

`LoanAccount` holds state and enforces its own invariants, and does nothing else — it neither computes schedules nor renders itself.

4.2 Open–Closed Principle — relevant, demonstrated in the design

The design decision: adding a fourth loan product must not require editing any tested class.

`LoanProduct` defines the stable `generateSchedule()` workflow and declares an abstract `createInterestCalculator()`. A new product is a new subclass plus a new calculator — two new files. `generateSchedule()` is never reopened, so the three existing products' tests cannot be broken by the arrival of a fourth.

The same shape appears in `origination`: eligibility is a list of `EligibilityRule` objects, so a new rule is a new class and a registration line, not an edit to `EligibilityEvaluator`.

What was rejected: a `switch (productType)` inside schedule generation. It would place every product's arithmetic in one method that must be reopened — and re-tested in full — for every new product, with the risk that a mistake in the new branch changes what existing borrowers are charged.

4.3 Liskov Substitution Principle — relevant, designed; not yet observable (M2)

The design decision, stated as something deliberately not done: suppose the institution later adds interest-free hardship loans (a fourth product, outside the M1 scope in §1.4). The tempting modelling is `HardshipLoan extends ReducingBalanceProduct` with `createInterestCalculator()` overridden to throw `UnsupportedOperationException`, since there is no interest to calculate.

That is rejected. It refuses an operation the supertype promises, so a `HardshipLoan` cannot stand in for a `LoanProduct`, and every caller would need `instanceof` to avoid the exception. Instead, a `HardshipProduct` would return a `ZeroInterestCalculator` from its factory method; that calculator honours the contract and returns `Money.ZERO`. Substitution holds, and no caller changes.

The contract every `InterestCalculator` honours: same preconditions (any non-negative principal, any tenure within the product band); returns a non-negative `Money`, never null; a schedule always has exactly `tenureMonths` instalments. These are stated in the interface's Javadoc and asserted in a **shared contract test** that every implementation must pass — so LSP is checked by the build rather than by good intentions.

4.4 Interface Segregation Principle — relevant, demonstrated in the design (code-level evidence at M2)

The design decision: the persistence boundary is not one fat `LoanDao`.

Separate role interfaces: `ScheduleProvider` (read a schedule), `LedgerWriter` (append an entry), `LedgerReader` (query entries), `RepaymentPoster` (post a repayment), `ReportExporter` (write CSV). One class may implement several roles — `SqliteLedgerRepository` implements both `LedgerWriter` and `LedgerReader` — but each client is handed only the role it needs (Views 2–4).

The concrete consequence: apart from `LoanRepository` for account status, the end-of-day engine receives `ScheduleProvider` and `LedgerWriter` only, and `reporting` receives `LedgerReader` only. The engine is therefore *incapable* of exporting a CSV or reading the full ledger, and a report is incapable of writing one — not by policy, but because those methods are not on the interfaces it holds. A fat interface would have made the engine's test double implement export methods it never calls, and every one of those stubs would be a latent failure waiting to be invoked.

4.5 Dependency Inversion Principle — relevant, demonstrated in the design (code-level evidence at M2)

The design decision: the domain declares the interfaces it needs; persistence implements them.

`LoanRepository`, `LedgerWriter` and `LedgerReader` are declared **inside the domain modules**, not in the persistence package. `SqliteLoanRepository` and `SqliteLedgerRepository` implement them and are injected at the composition root — the one place in the application that is allowed to call `new` on infrastructure.

The concrete consequence: no business class imports `java.sql`. Unit tests inject in-memory fakes and run with no database present. A ledger-posting test runs in milliseconds and cannot fail because of a locked file or a missing driver.

What was rejected: `new SqliteLoanRepository(connection)` inside `CoreRepaymentPoster`. That would make the posting logic untestable without a real database, push a JDBC concern into business code, and make a storage change a business-code change.

4.6 Summary and limits

Principle	Status at M1	Concrete design decision	Evidence that will show it
SRP	Relevant — demonstrated in the design	Splitting of <code>LoanService</code> into evaluator, product/assembler, allocator, renderer — §4.1	Visible now in the class views
OCP	Relevant — demonstrated in the design	<code>LoanProduct</code> hierarchy and the <code>EligibilityRule</code> list — §4.2	Visible now in View 1
LSP	Relevant — designed, not yet observable	Hardship lending modelled as a zero calculator, plus the shared contract test — §4.3	Contract test passing for all three calculators (M2)

ISP	Relevant — demonstrated in the design	Role interfaces at the persistence and posting boundary — §4.4	Visible now in Views 2–4; one small test double per role (M2)
DIP	Relevant — demonstrated in the design	Domain-declared repository interfaces, injected implementations — §4.5	Visible now in Views 2–4; suite runs with in-memory fakes and no database (M2)

M1 commits to the structure; M2 produces the code-level evidence for all five, and the first observation of LSP.

Where a principle is weaker. SRP at the *module* level is clean, but `RepaymentAllocator.allocate()` is a single method with a cyclomatic complexity of about 11 (ten decision points), which is the most complex method in the system. It is kept as one method deliberately — splitting the waterfall across classes would scatter a rule that must be read in order — and it is the primary subject of the Fagan inspection and the basis-path test design on the SQA side.

5 Design Pattern Selection and Justification

Three patterns — one creational, two structural — each chosen because a construction problem came first and the pattern second.

5.1 Factory Method — `LoanProduct` as Creator (creational)

Owner: Hassan Khalid

The construction problem. `generateSchedule()` is a stable workflow: validate the tenure, build the due-date series, compute each instalment's interest, absorb the rounding residue into the final instalment, assemble the schedule. Exactly **one** step varies — which interest calculation applies. If the workflow writes `new ReducingBalanceCalculator()`, the schedule module becomes coupled to concrete classes through the `new` keyword, and must be reopened for every new product.

The structure. `LoanProduct` is the **Creator** and holds the stable workflow. `createInterestCalculator()` is the **factory method**, declared abstract by the Creator and overridden by `FlatRateProduct`, `ReducingBalanceProduct` and `BulletProduct` — the **ConcreteCreators**. `InterestCalculator` is the **Product** abstraction, and the three calculators are **ConcreteProducts**. The creator subtype determines what is created; the workflow never names a concrete class.

```
public abstract class LoanProduct {
    public final Schedule generateSchedule(Money principal, int tenureMonths, LocalDate start) {
        InterestCalculator calculator = createInterestCalculator(); // ← the factory method
        // ... stable workflow: dates, per-instalment interest, rounding residue, assembly
    }
    protected abstract InterestCalculator createInterestCalculator();
}
```

Alternative rejected: Simple Factory. A `ScheduleFactory.create(type)` holding a `switch` was the obvious first design. It is rejected because the creation decision stays centralised in one place that must be edited for every new product — it *moves* the conditional ladder rather than removing it. A method that returns a new object is not automatically the GoF Factory Method; the distinguishing property is that instantiation is **deferred to subclasses** overriding the creator's factory method, rather than decided by a parameter passed to one central method.

Also rejected: Abstract Factory, because there is no product *family* here — only one axis of variation, so a second factory layer is indirection without a second dimension. **Also rejected: Singleton** for a shared product registry, because it hides dependencies, introduces global mutable state and leaks state between tests; constructor injection already gives one shared instance without the coupling.

5.2 Adapter — the bank statement adapters (structural)

Owner: Muhammad Ibrahim (package `ledger.importing`)

The construction problem. Repayments also arrive as bank statement files, and every bank's format differs: column order, date format, and amount convention — several report amounts in **minor units** (paisa) as integers. Files are read with Apache Commons CSV, a third-party library whose `CSVParser` yields rows of untyped strings (`CSVRecord`). That interface was designed independently of ours and we cannot change it. The servicing core must keep thinking in `Money` and `LocalDate`.

The structure. The **Client** is the repayment-posting workflow. The **Target** is `RepaymentSource`, the interface our application already understands. The **Adaptee** is the library's `CSVParser`. The **Adapters** — `HblStatementAdapter` and `McbStatementAdapter`, one per supported bank format — read rows through the parser, convert `254900` into `Money.of(new BigDecimal("2549.00"))` and `04102026` into a `LocalDate`, and translate the parser's failures into `InvalidRepaymentException`. Each is an **Object Adapter**: it *has-a* parser rather than inheriting one. Adding a bank is a new adapter class; the posting workflow does not change.

Alternative rejected: Facade. The distinction is intent: an Adapter protects clients from an **interface mismatch**; a Facade protects clients from **excessive subsystem knowledge**. At the import boundary there is no complicated subsystem to simplify — there is exactly one foreign contract that does not fit ours. That is mismatch, not complexity.

Also rejected: changing the core to speak the bank's format. That moves the dependency in the wrong direction, spreading vendor vocabulary — minor units, their date format, their sign convention — through business logic, which is precisely the coupling the Adapter exists to contain at one boundary.

5.3 Decorator — the `RepaymentPoster` pipeline (structural)

Owner: Tughrul Hussain

The construction problem. Posting a repayment must sometimes also (a) write an audit journal line naming the operator, (b) record elapsed time for the GQM measurement programme, and (c) guard against posting the same bank reference twice. **Which of these are needed differs per call path, and they combine:**

Call path	Audit	Timing	Idempotency
Teller screen	🔊	🔊	—
CSV bulk import	— (the import job writes one audit line per file)	🔊	🔊
End-of-day internal adjustment	—	🔊	—
Unit tests	—	—	—

The structure. `RepaymentPoster` is the **Component**; `CoreRepaymentPoster` is the concrete component holding the real allocation call; `RepaymentPosterDecorator` both **IS-A** `RepaymentPoster` and **HAS-A** `RepaymentPoster`. `AuditLoggingPoster`, `TimingPoster` and `IdempotencyGuardPoster` each add one focused behaviour and then delegate to `wrapped.post(c)`.

Composition order is deliberate and documented: wherever the idempotency guard is used, it wraps outermost, so a duplicate is rejected before anything is timed or posted. Order matters in a Decorator chain, and stating why is part of the design. The two production chains, built at the composition root:

```

RepaymentPoster importPoster =           // CSV bulk import
    new IdempotencyGuardPoster(           // outermost: a duplicate stops here
        new TimingPoster(
            new CoreRepaymentPoster(allocator, settlementPolicy,
                                    loanRepository, ledgerWriter)));

RepaymentPoster tellerPoster =           // teller screen: no bank reference to guard
    new AuditLoggingPoster(
        new TimingPoster(
            new CoreRepaymentPoster(allocator, settlementPolicy,
                                    loanRepository, ledgerWriter)));

```

Alternative rejected: inheritance combinations. `AuditingPoster`, `TimingPoster`, `AuditingTimingPoster`, `AuditingIdempotentPoster` ... — eight classes for three optional behaviours, and sixteen the day a retry behaviour is added. This is the classic subclass explosion, and the combinations cannot be chosen at runtime per call path.

Also rejected: Proxy. The two have nearly the same structure and are separated by intent: a Decorator primarily **extends behaviour**; a Proxy primarily **controls access** to one subject. Taken alone, `IdempotencyGuardPoster` does gate — it refuses a duplicate — and a protection Proxy could do that one job. The design problem, though, is not one access policy in front of one object: it is three optional responsibilities that must **stack in different combinations per call path**, chosen at composition time (table above). That is Decorator's pressure, not Proxy's, and the guard is simply one layer in the stack.

5.4 Patterns considered and deliberately not used at M1

Pattern	Why not now
Facade — <code>RepaymentFacade.postRepayment()</code>	A genuine candidate: three clients need the same five-step workflow. Held as a possible fourth pattern at M2. If adopted, it must coordinate and not absorb — the allocation rules stay in <code>RepaymentAllocator</code> , or it becomes a god class
Strategy — <code>ProvisioningPolicy</code>	Valid, and likely at M2 for swapping provisioning percentages when regulations change. Not claimed at M1, and <code>InterestCalculator</code> will not be presented as a Strategy, because it is the Product of the Factory Method above; the calculators do play a Strategy role, but counting the same classes as a second pattern would inflate the pattern count
State — loan lifecycle; Observer — classification alerts	Both fit the domain, and both are deferred to M2 rather than adopted on vocabulary the team cannot yet defend in depth

A pattern is only included where it solves a problem that existed in the design before the pattern was named. The three above each do.

6 Repository and Documentation Setup

Repository: <https://github.com/tughra1/microlend> (public)

Collaborators: `tughra1`, `mIBRAHIM707`, `Hassan242-kk`

6.1 Branching strategy

Three tiers:

Branch	Purpose	Rules
<code>main</code>	Milestone baselines only; the default branch	Protected by the ruleset below. No direct pushes. Changes arrive solely by reviewed pull request from <code>develop</code>
<code>develop</code>	Integration branch, the current working state	Receives pull requests from feature branches; each is reviewed by a member other than the author
<code>feature/<module>-<short-name></code>	One unit of work, owned by one member	Branched from <code>develop</code> , merged back by pull request

6.2 Quality gates

A GitHub **ruleset** (`main-protection`) is active on `main` :

- Direct pushes are **blocked** — every change requires a pull request
- **One approving review** required, from someone other than the author
- Stale approvals are **dismissed** when new commits are pushed
- **Last-push approval** required, so code cannot be slipped in after approval
- All review conversations must be **resolved** before merge
- Merge commits are the only allowed merge method
- Branch deletion and force-pushes are blocked
- No bypass actors are configured, so the rules apply to every member, including the repository owner

`CODEOWNERS` automatically requests review from the owner of the module a pull request touches; for shared code — `Money`, the exception hierarchy, the Checkstyle configuration — it requests all three members.

Note: the M1 scaffold (documents, build, CI, `Money`) was committed directly to `develop` while the repository was being set up, before any feature work existed. From M2, every change to `develop` arrives by pull request from a `feature/` branch, and the same ruleset is extended to `develop` when the first feature branch opens.

6.3 Preserving the TDD trail

Squash-merge and rebase-merge are disabled in repository settings; merge commits are the only permitted method. This is deliberate: squashing collapses a red-green-refactor sequence into a single commit, and the evidence cannot be reconstructed afterwards.

Commit convention:

```
test(red):      instalment rounding residue lands on the final instalment
feat(green):    absorb rounding residue in ScheduleAssembler
refactor:       extract RoundingPolicy from ScheduleAssembler
```

The pull-request template requires the author to name the red, green and refactor commit hashes for the change.

6.4 Baselines and CI

Milestone submissions are tagged on `main` : `m1-baseline` (Week 6), `m2-baseline` (Week 10) and `m3-baseline` (Week 13). A tag is created only at submission, so it points at exactly the state that was assessed.

The CI pipeline (GitHub Actions, `.github/workflows/build.yml`) runs on every push to `main`, `develop` and `feature/**`, and on every pull request into `main` or `develop` : compile → JUnit → Checkstyle gate → JaCoCo coverage report, with the reports uploaded as build artifacts. It requires no secrets and no network access beyond fetching dependencies. It is already running and green on `develop`.

6.5 Coding standards

`config/checkstyle.xml` is the project coding standard, and it enforces **constraint C-01: money is never a binary floating-point type**. `double` and `float` are rejected by the build, and a second rule catches stray decimal literals such as `0.05`. All monetary values use `BigDecimal` inside `Money` with an explicit `RoundingMode`. `0.1 + 0.2 ≠ 0.3` in IEEE 754, and a fraction of a rupee per instalment accumulates across a portfolio into a ledger that does not balance.

The configuration also enforces complexity limits (cyclomatic complexity ≤ 15 per method, source files ≤ 400 lines), rejects empty catch blocks and `catch (Exception)`, and forbids public mutable fields.

6.6 Documentation

File	Contents
<code>README.md</code>	System overview, module table with owners, build and run instructions, branching strategy, commit convention, coding standards
<code>docs/M1-SE423-*.md</code>	This document
<code>docs/M1-SE431-*.md</code>	The companion SE431 quality document
<code>docs/business-rules.md</code>	The Business Rules Register — every rule frozen at M1, with a deferred list
<code>qa/rtm.md</code>	Requirements Traceability Matrix, version 1
<code>qa/risk-register.md</code>	Living risk register, initiated at M1 with 8 scored risks
<code>qa/m1-ai-usage-log.md</code>	AI Usage Log for this milestone; M2 and M3 have their own
<code>config/checkstyle.xml</code>	The coding standard, referenced by SQAP §4.4

Annex A — AI Usage Log

Required by the joint-project guidelines §9. Milestones 2 and 3 carry their own logs.

Every entry used Claude through Claude Code. Each entry is dated the day the work was done, which the repository's commit history corroborates.

#	Date	Tool	Purpose	Prompt / activity summary	Material adopted	Modifications made	Verification
1	2026-10-04	Claude Opus 5	Compare candidate project ideas against both rubrics	Asked for candidate project ideas scored against the §3.1 and §3.2 minimum requirements	Comparative analysis; MicroLend chosen over two alternatives	Scope cut to a locally-run application with no deployment or paid services	Each rubric minimum traced to where it is met; team accepted the choice
2	2026-10-04	Claude Opus 5	Draft the SE423 document	Asked for a class model, SOLID analysis and pattern justifications from the module list and business rules	Class model, SOLID analysis, pattern justifications	Diagram split into views for legibility; CLI replaced by a local web interface	Found and corrected a Simple Factory presented as a Factory Method
3	2026-10-04	Claude Opus 5	Draft the SE431 document	Asked for ISO 25010 quality requirements, an RTM, an IEEE 730 outline, a CoQ estimate and a risk register for the SE423 design	Quality requirements, RTM, SQAP outline, CoQ, risk register	Four items reclassified from quality requirements to constraints	Targets and effort estimates checked as realistic for a three-person team
4	2026-10-04	Claude Opus 5	Repository setup and coding standard	Asked for a Checkstyle configuration enforcing C-01, a CODEOWNERS file, a PR template and branch rules	Checkstyle config, CODEOWNERS, PR template, branch rules	C-01 check rewritten as a token rule after a false positive	Branch protection tested by a direct push, which was blocked and logged
5	2026-10-04	Claude Opus 5	Business Rules Register	Asked to turn each module's rules into numbered rules with boundary values for test design	43 numbered rules with boundary values	Numeric thresholds reviewed and accepted unchanged by the team	Boundary values cross-checked against the RTM test design

6	2026-10-04	Claude Opus 5	Build and CI scaffold	Asked for a Gradle build, a failing <code>MoneyTest</code> first, then <code>Money</code> , and a GitHub Actions workflow	Gradle build, <code>Money</code> , <code>MoneyTest</code> , GitHub Actions workflow	Written as a real TDD cycle, red commit before green	CI green: compile, 11 tests, Checkstyle, coverage
7	2026-10-04	Claude Opus 5	Submission PDFs	Asked for a pandoc/LaTeX script producing both submission PDFs with a title page	Build script and title page	Fonts changed to carry mathematical symbols	Pages inspected; a fault dropping <code>≤</code> and <code>×</code> was found and fixed
8	2026-10-07	Claude Opus 5.5	Pre-submission review of both documents and the repository	Asked to check both M1 documents against the joint guidelines and rubrics, list inconsistencies, then apply the fixes	Corrections to both documents, RTM, risk register, business rules, README and CODEOWNERS: M1 Team Lead, class views matched to the text and RTM, SOLID status labels, milestone references matched to the updated guidelines, an independent fact-check pass, PDFs rebuilt with an HTML/CSS pipeline (Paged.js)	M1 Team Lead confirmed by the team; an interim four-milestone revision was reverted to three milestones when the instructor issued the updated guidelines the same day; risk-grade assignment and active-loan counting defined at the team lead's request and confirmed by all three members (BR-03, BR-09)	Each change checked against the guidelines PDF and the live repository (ruleset, CI runs, <code>Money.java</code>); PDFs rebuilt and every page inspected

Declaration for Milestone 1

Each member named below has read, understood and verified the material credited to them, can explain and defend the design decisions, quality requirements and analysis it contains, and reviewed and modified any AI-assisted content where needed.

- **Muhammad Ibrahim — 2023446** · `origination`, `delinquency`; SQAP outline, risk register
- **Hassan Khalid — 2023242** · `products`, `reporting`; quality requirements, RTM
- **Tughrul Hussain — 2023532** · `ledger`; standards awareness, Cost of Quality, repository setup

Team rules for AI use

1. Generated code enters the repository only through the module owner, who reviews it and answers for it.
2. No generated test is accepted without reading what it asserts.
3. Business rules and numeric thresholds are the team's decisions, not defaults.
4. Entries are added when the work happens, not before a deadline.
5. Anything a member cannot explain is rewritten or withdrawn before submission.